

Concurrency and Distributed Systems

Week 8 – Part 2: Broken Async Server and Fix

Dr. Mohamad Aoude

Faculty of Engineering

May 13, 2026

Part 2 Roadmap

1 The Broken Server

2 The Fix

3 Demo

Broken Completion Path

```
1 CompletableFuture<String> dependency =  
2     new CompletableFuture<>();  
3  
4 return dependency.join();
```

Failure

If no one completes the dependency future, the caller can wait forever.

Mechanism First

- Symptom: request hangs.
- Evidence: request worker waits in `CompletableFuture.join`.
- Mechanism: dependency future has no completion, timeout, or fallback path.
- Fix: define bounded waiting before the caller waits.

Timeout Fallback

```
1 CompletableFuture<String> dependency =  
2     new CompletableFuture<>();  
3  
4 return dependency.completeOnTimeout(  
5     "fallback-response",  
6     timeout.toMillis(),  
7     TimeUnit.MILLISECONDS);
```

Meaning

The request no longer waits forever. It receives a controlled degraded response after the timeout.

Fix Quality

A good fix:

- addresses the mechanism,
- bounds waiting,
- exposes fallback behavior,
- does not hide the operational cost,
- is easy to test deterministically.

Run

```
.\run-demo.ps1 03
```

- Show the broken state.
- Show the fixed fallback response.
- Map code back to thread-dump evidence.